

B32K

Open Specification for Distributed Symbolic Systems

v1.0.0

Marcel Kantimm
T-Esoist Technology

Scott Allen Cave
ORCID: [0009-0000-0888-4232](https://orcid.org/0009-0000-0888-4232)

Joachim Kiseleczuk
Atlan-Code Cooperative

Stóló ∴ Terra Firma ∴ Gaia

Scott Allen Cave ∴ **Gizmo**

Joachim Kiseleczuk ∴ **Heliathon**

Marcel Kantimm ∴ **Aether 3.0**

Published in: Communications of Recursive Inquiry

DOI: [10.5281/zenodo.17478620](https://doi.org/10.5281/zenodo.17478620)

Released in Stóló Time

29 > 29 October 2025

Abstract

The B32K Open Specification defines a canonical base-32,768 alphabet and an associated message-envelope format for secure interchange of structured data. Each symbol encodes fifteen bits and maps to an immutable Unicode code point through a deterministic lookup table. A strict canonicalization procedure $N(x)$ combines UTF-8 NFC normalization with canonical CBOR (RFC 8949) to ensure byte-for-byte reproducibility across implementations.

Transport security is provided through the **HTTPQ** envelope, which authenticates metadata (the Public Sector) while encrypting payloads (the Private Sector) under a modern cryptographic suite (Ed25519, X25519, HKDF-SHA256, ChaCha20-Poly1305, BLAKE3) with optional post-quantum hybrid extensions (ML-KEM + ML-DSA). A deterministic nonce-derivation mechanism and a structured **Anchor** / **Ledger** architecture enable end-to-end auditability, versioning, and domain separation.

Normative sections specify canonical encoding, bit-packing of 15-bit indices, key-derivation labels, and AEAD verification semantics. Appendices provide test vectors, registry identifiers, compliance mappings, and research extensions such as the Planar Permutation Layer (PPL) for quantum-phase security.

The specification is suitable for cryptographic, financial, medical, and ledger-based systems requiring deterministic serialization, verifiable integrity, and long-term interoperability.

Contents

1	Normative References	4
2	Definitions and Symbols	4
2.1	2.1 Core Concepts	4
2.2	2.2 Canonicalization $N(x)$	5
2.3	2.3 Bit-Packing of 15-bit Indices	5
2.4	2.4 Constants	5
3	CBOR Encoding Rules	5
3.1	3.1 Algorithm	5
3.2	3.2 Example	6
4	Cryptographic Suites	6
4.1	4.1 Baseline Suite (Classical 2025)	6
4.2	4.2 Post-Quantum Hybrid Suite	7
4.3	4.3 Planar Permutation Layer (PPL)	7
4.4	4.4 Key Lifecycle Management	8
5	HTTPQ Transport	8
5.1	5.1 Public Sector (AAD)	8
5.2	5.2 Private Sector	10
5.3	5.3 Build / Verify	10
6	Ledger Notarization	11
6.1	6.1 Handle and Digest Construction	11
6.2	6.2 Merkle Aggregation	11
6.3	6.3 Anchoring to a Ledger	11
6.4	6.4 Receipts and Proof Objects	12
6.5	6.5 Cross-Domain and NFT Integration (Non-Normative)	12
6.6	6.6 Security and Audit Notes	12
7	Security Considerations	13
7.1	7.1 Key Management and Separation	13
7.2	7.2 Randomness and Nonce Handling	13
7.3	7.3 Replay Protection and Freshness	13
7.4	7.4 Hashing and Integrity Protection	13
7.5	7.5 Glyphs, Fonts, and Human Interfaces	14
7.6	7.6 Post-Quantum Transition	14
7.7	7.7 Implementation Best Practices	14
8	Compliance Mapping	14
9	Compliance Mapping	14
9.1	8.1 Standards Alignment Overview	15
9.2	8.2 Privacy and Minimal Disclosure	15
9.3	8.3 Security Transport and Lifecycle Controls	15
9.4	8.4 Assurance Summary	15

A	Canonicalization Test Vectors	16
A.1	A.1 Scalar Examples	16
A.2	A.2 Structured Object Example	16
A.3	A.3 15-bit Packing Reference	16
A.4	A.4 Signature Pre-image Template	17
A.5	A.5 Conformance Validation Pseudocode	17
A.6	A.6 Deterministic Output Property	17
B	Cryptographic Suite Identifiers & Key Derivation Labels	17
B.1	B.1 Algorithm Identifier Registry	17
B.2	B.2 HKDF Domain-Separation Labels	18
B.3	B.3 Deterministic Nonce Derivation	18
B.4	B.4 Key-Separation Example	19
B.5	B.5 Registry Management	19
C	O-1 Receipt Schema and Merkle Proof Examples	19
C.1	C.1 JSON Schema Definition	19
C.2	C.2 Example Receipt	20
C.3	C.3 Verification Pseudocode	20
C.4	C.4 Error Codes	20
C.5	C.5 NFT and Cross-Ledger Verification (Non-Normative)	21
C.6	C.6 Deterministic Properties	21
D	Planar Permutation Layer (PPL)	21
D.1	D.1 Formal Model	21
D.2	D.2 Implementation Parameters	22
D.3	D.3 Authenticated Context	22
D.4	D.4 Example Pseudocode	22
D.5	D.5 Quantum-Phase Security Research Path (Non-Normative)	23
D.6	D.6 Summary	23
E	Glossary and Symbol Index	23
E.1	E.1 Core Terms	23
E.2	E.2 Symbols and Constants	24
E.3	E.3 Abbreviations	24
E.4	E.4 Notational Conventions	25
E.5	E.5 Cross-Reference Index	25
F	Safe Rendering Plane (Non-Normative)	25
G	HTTPQ Envelope Error Codes	26
G.1	G.1 Error Registry (Normative)	26
G.2	G.2 Verification Decision Flow (Informative)	28
G.3	G.3 Mapping to Telemetry (Informative)	28

1 Normative References

This specification relies upon the following external standards and publications. All are considered **normative** unless otherwise stated.

Reference	Title / Description
RFC 8949	<i>Concise Binary Object Representation (CBOR)</i>
RFC 8785	<i>JSON Canonicalization Scheme (JCS)</i>
RFC 4648	<i>Base Encodings for Binary Data</i>
Unicode 15.1	<i>The Unicode Standard v15.1 (2023)</i>
NIST SP 800-53 Rev. 5	<i>Security and Privacy Controls</i>
NIST SP 800-161 Rev. 1	<i>Cyber Supply Chain Risk Management Practices</i>
NIST PQC Round 4 Drafts	<i>ML-KEM (Kyber) and ML-DSA (Dilithium) Candidates</i>
BLAKE3 Specification	<i>BLAKE3: One Function, Fast Everywhere</i>
Ed25519 (RFC 8032)	<i>High-Speed, High-Security EdDSA Signatures</i>
ChaCha20-Poly1305 (RFC 8439)	<i>Authenticated Encryption with ChaCha20 and Poly1305</i>
PCI DSS v4.0	<i>Payment Card Industry Data Security Standard</i>
HIPAA Security Rule	<i>45 CFR Part 164 Subpart C — Security of ePHI</i>
GDPR (EU 2016/679)	<i>General Data Protection Regulation</i>
SLSA v1.0	<i>Supply-chain Levels for Software Artifacts Framework</i>

2 Definitions and Symbols

All RFC 2119 keywords (**MUST**, **SHOULD**, **MAY**) are to be interpreted as normative requirements.

2.1 Core Concepts

Index (15-bit). Unsigned integer in the inclusive range [0, 32767]. Represents the authoritative identity of a symbol; glyph renderings are advisory only.

Glyph. Advisory Unicode representation of an index. Glyphs **MUST NOT** carry cryptographic meaning or affect canonicalization.

Alphabet Table. Canonical mapping

$$A : \text{Index} \rightarrow \text{CodePoint},$$

defined in Appendix ???. The 15-bit index is normative; the Unicode code point is a rendering convenience.

Lane. Unsigned 16-bit integer used as a domain-separation identifier for independent protocol contexts.

Version. Semantic version string included in all signatures and normalization contexts. Follows the pattern MAJOR.MINOR.PATCH.

Anchor. Structured record containing version, lane, checksum, and optional digital signatures. Anchors **MUST** be serialized using the canonical CBOR rules defined in Section 3.

2.2 Canonicalization $N(x)$

The canonicalization function $N(x)$ produces a unique byte representation of any structured object x :

$$N(x) = \text{CBOR_Canonical}(\text{UTF8_NFC}(x)).$$

Text fields **MUST** be normalized to Unicode NFC prior to encoding. CBOR maps **MUST** use definite lengths and canonical key ordering.

2.3 Bit-Packing of 15-bit Indices

Each 15-bit index is encoded in big-endian order and concatenated bitwise into a continuous bitstream. Padding with zero bits to the nearest octet boundary **MUST** be applied at the end of the stream.

2.4 Constants

Symbol	Description
B32K_INDEX_MAX	32767 — Maximum canonical index value.
B32K_LANE_MAX	65535 — Maximum allowable lane identifier.
$N(x)$	Canonicalization output in bytes.

3 CBOR Encoding Rules

Canonicalization in B32K combines Unicode NFC normalization with canonical CBOR encoding as defined in RFC 8949. The resulting byte sequence, denoted $N(x)$, is the sole authoritative representation used for cryptographic signatures and checksums:

$$N(x) = \text{CBOR_Canonical}(\text{UTF8_NFC}(x)).$$

3.1 Algorithm

Algorithm 1 Canonicalization $N(x)$

Require: Input object x

Ensure: Canonical byte sequence b

- 1: $y \leftarrow \text{UTF8_NFC}(x)$
 - 2: Encode y with CBOR, enforcing:
 - Definite-length arrays and maps only;
 - Shortest possible integer representations;
 - Floating-point values encoded as 64-bit binary64;
 - Map keys sorted by their encoded CBOR byte order;
 - No duplicate map keys.
 - 3: $b \leftarrow \text{CBOR_Encode}(y)$
 - 4: **return** b
-

3.2 Example

Canonical CBOR hex encoding for:

```
{"lane":4, "version":"1.1.0", "payload":[3,5,32767]}
```

is:

A3	# map(3)
64 6C616E65	# "lane"
04	
67 7061796C6F6164	# "payload"
83 03 05 19 7FFF	# array(3)
67 76657273696F6E	# "version"
65 312E312E30	# "1.1.0"

The corresponding canonical hex stream (no whitespace) is:

```
A3646C616E6504677061796C6F6164830305197FFF6776657273696F6E65312E312E30
```

Verification. A conforming implementation **MUST** produce byte-identical output for this object regardless of programming language, host platform, or locale settings.

Error Conditions.

- Indefinite-length maps or arrays **MUST NOT** appear.
- Duplicate keys **MUST** cause rejection.
- Non-UTF-8 or non-NFC text **MUST** be normalized prior to encoding.
- Encoders **MUST** reject NaN or non-finite float encodings.

4 Cryptographic Suites

All cryptographic operations in this specification are performed over the canonical byte representation $N(x)$ (see Section 3). This ensures that signatures, checksums, and digests are stable and platform-independent.

4.1 Baseline Suite (Classical 2025)

The baseline B32K suite provides strong, efficient, and widely supported classical security. It is suitable for general-purpose deployments where post-quantum assurances are not required.

Function	Algorithm
Signature	Ed25519 (RFC 8032)
Key Exchange (KEX)	X25519 (RFC 7748)
Key Derivation	HKDF-SHA256 (RFC 5869)
Authenticated Encryption (AEAD)	ChaCha20–Poly1305 (RFC 8439)
Hash / Integrity	BLAKE3-256 (BLAKE3 Specification)

Operational Requirements.

- Each key role **MUST** use an independent keypair or derivation.
- Nonces **MUST** be unique per message; 96-bit random or deterministic nonces are acceptable.
- Keys **MUST** be derived via HKDF using explicit domain-separation labels (see Appendix B).
- Reuse of Ed25519 and X25519 key material across signing and KEX roles is **forbidden**.

4.2 4.2 Post-Quantum Hybrid Suite

The hybrid suite combines classical and post-quantum primitives to provide security against both current and future adversaries.

Function	Hybrid Composition
Key Exchange	X25519 ML-KEM (Kyber)
Signature	Ed25519 ML-DSA (Dilithium)
Key Derivation	HKDF-BLAKE3
AEAD	ChaCha20-Poly1305

Composition Rule. Shared secrets from both classical and PQC algorithms are concatenated:

$$\text{IKM} = \text{X25519_shared} \parallel \text{ML-KEM_shared}.$$

The combined secret is expanded using HKDF-BLAKE3 with explicit hybrid labels: "B32K-KEX-Hybrid" for key exchange and "B32K-SIG-Hybrid" for signatures.

Normative Guidance.

- The hybrid suite **MUST** be explicitly negotiated via the `alg_id` field (Appendix B).
- PQC algorithms **MUST** follow parameter sets aligned with NIST Round 4 drafts.
- Implementations **MUST** reject truncated or mismatched concatenations.

4.3 4.3 Planar Permutation Layer (PPL)

The **Planar Permutation Layer (PPL)** is an optional, research-grade pre-encryption transformation designed to obfuscate payload structure and simulate phase-space encoding.

Transform. Given the canonical byte stream $N(x)$:

1. Split $N(x)$ into n contiguous *planes*.
2. Derive a permutation key K_p using HKDF with the label "B32K-PPL".
3. Generate permutation σ and spin vector ω_i from a keyed PRNG stream seeded by K_p .
4. Apply the rotation-permutation transform:

$$p'_i = \text{rotate}_{\omega_i}(p_{\sigma(i)}).$$

5. Concatenate the transformed planes to form the input to the AEAD layer.

Decryption applies the inverse transform $p_i = \text{rotate}_{-\omega_i}(p'_{\sigma^{-1}(i)})$. Full mathematical formalism is provided in Appendix D.

Usage. When enabled, the PPL operates between canonicalization and encryption:

$$N(x) \xrightarrow{\text{PPL}} N'(x) \xrightarrow{\text{AEAD}} C.$$

Implementations **MUST** ensure the outer AEAD tag authenticates the transformed payload in its entirety.

4.4 Key Lifecycle Management

- Session keys **MUST** rotate every 24 hours or after 2^{20} messages, whichever occurs first.
- Ledger anchoring keys and transport session keys **MUST** be derived separately.
- Persistent signing keys (Ed25519 or ML-DSA) **SHOULD** be protected by hardware security modules (HSMs) or equivalent.

5 HTTPQ Transport

The HTTPQ transport provides a versioned, authenticated envelope for B32K payloads. Each envelope consists of two distinct components:

- **Public Sector (AAD)** — metadata visible to routing and verification layers but not user payloads.
- **Private Sector (Ciphertext + Tag)** — AEAD-encrypted body, optionally preceded by the Planar Permutation Layer (PPL).

The envelope design separates these concerns for clarity and cryptographic composability.

5.1 Public Sector (AAD)

The AAD (Additional Authenticated Data) contains all non-secret metadata used to authenticate and route packets. Fields are:

- `version` — Specification version string (semantic).
- `lane_id` — 16-bit domain-separation identifier.
- `alg_id` — Algorithm suite identifier.
- `timestamp` — UTC epoch seconds.
- `packet_id` — 128-bit UUIDv7 identifying the envelope.
- `seq_no` — 64-bit sequence number.
- `nonce` — 96-bit nonce for AEAD (deterministic or random).
- `handle` — 32-byte digest binding metadata to payload.
- `payload_len` — unsigned 32-bit payload length in bytes.
- `ledger_ref` — optional ledger URI or digest reference.

All fields are encoded as canonical CBOR (RFC 8949) and supplied as AAD to the AEAD construction.

Handle. The handle is computed as:

$$\text{BLAKE3}_{256}(\text{"B32K-HANDLE"} \parallel \text{version} \parallel \text{lane_id} \parallel \text{BLAKE3}_{256}(N(x)))$$

where $N(x)$ is the canonicalized payload (§ 3). This binds the payload and envelope meta-data into a unique, reproducible digest.

AAD Schema (Normative)

Each AAD field is encoded using CBOR major/minor types in canonical form (definite lengths, shortest integer representation, UTF-8 text for strings).

Field	CBOR Type	Description / Encoding Notes
version	text string	Semantic version string, e.g. "1.1.0". UTF-8 NFC, no null termination.
lane_id	uint (major 0)	16-bit unsigned integer, big-endian.
alg_id	uint (major 0)	16-bit unsigned integer, big-endian.
timestamp	uint (major 0)	Seconds since Unix epoch UTC.
packet_id	byte string (major 2)	128-bit UUIDv7 binary (16 bytes).
seq_no	uint (major 0)	64-bit unsigned integer, big-endian.
nonce	byte string (major 2)	96-bit (12 bytes) AEAD nonce (App. B B.3).
handle	byte string (major 2)	32-byte BLAKE3-256 digest (§ 6).
payload_len	uint (major 0)	32-bit unsigned integer (length of plaintext).
ledger_ref	text string (opt)	Optional URI or hash reference to ledger record (§ 6).

All AAD elements are aggregated into a single CBOR map with lexicographically sorted key order before AEAD initialization. Example canonical order:

```
{"alg_id":..., "handle":..., "lane_id":..., "ledger_ref":..., "nonce":..., "packet_id":..., "payload_len":..., "seq_no":..., "timestamp":..., "version":...}.
```

Canonical AAD Example (Informative)

Example AAD object (before CBOR encoding):

```
{
  "alg_id":      0x0101,
  "handle":      h'7CDA4F01D6C3E9124F5C63B1F9A9A96E2A89ABF9407B5EBA2EAF0FBCFEEB4A12',
  "lane_id":      4,
  "ledger_ref":   "urn:ledger:b32k:1.0.0",
  "nonce":        h'74BE16979710D4C4CEFE8D7ED2DC5594',
  "packet_id":    h'017F8A22C2E94D04A0C8B1A7AC2EED4F',
  "payload_len":  1024,
  "seq_no":        42,
  "timestamp":    1730209200,
  "version":      "1.1.0"
}
```

After CBOR canonical encoding (RFC 8949 definite-length map, key order sorted by encoded bytes), the byte layout is:

A9

```

67 61 6C 67 5F 69 64          # "alg_id"
19 01 01
66 68 61 6E 64 6C 65          # "handle"
58 20 7CDA4F01D6C3E9124F5C63B1F9A9A96E2A89ABF9407B5EBA2EAF0FBCFEEB4A12
67 6C 61 6E 65 5F 69 64      # "lane_id"
04
6A 6C 65 64 67 65 72 5F 72 65 66
74 75 72 6E 3A 6C 65 64 67 65 72 3A 62 33 32 6B 3A 31 2E 30 2E 30
65 6E 6F 6E 63 65            # "nonce"
58 0C 74BE16979710D4C4CEFE8D7ED2DC5594
69 70 61 63 6B 65 74 5F 69 64 # "packet_id"
50 017F8A22C2E94D04A0C8B1A7AC2EED4F
6B 70 61 79 6C 6F 61 64 5F 6C 65 6E
19 04 00
67 73 65 71 5F 6E 6F          # "seq_no"
18 2A
69 74 69 6D 65 73 74 61 6D 70 # "timestamp"
1A 6711E9B0
67 76 65 72 73 69 6F 6E      # "version"
63 31 2E 31 2E 30

```

Canonical hex concatenation:

A967616C675F69641901016668616E646C6558207CDA4F01D6C3E9124F5C63B1F9A9A96E2A89ABF9407B5EBA2E

The encoded map size is 245 bytes. Implementations verifying HTTPQ envelopes **MUST** reproduce this byte-for-byte layout when re-computing AAD for AEAD tag verification or handle derivation (§ 6).

5.2 Private Sector

The Private Sector comprises the ciphertext and AEAD tag. The canonical AEAD construction is ChaCha20-Poly1305 or the hybrid post-quantum equivalent (see § 4). TLS 1.3 **MUST** be used in transit for confidentiality and integrity. The optional PPL transformation (§ D) occurs before AEAD encryption.

5.3 Build / Verify

Implementations **MUST** perform the following operations:

Algorithm 2 HTTPQ Build

Require: canonical payload bytes $b = N(x)$

Require: metadata fields per AAD schema

- 1: compute handle $\leftarrow$ BLAKE3-256("B32K-HANDLE" || version || lane_id || BLAKE3-256(b))
 - 2: assemble AAD (CBOR canonical map)
 - 3: derive nonce and keys (§ B)
 - 4: encrypt b with AEAD(ChaCha20-Poly1305, key, nonce, AAD)
 - 5: output {AAD, ciphertext, tag}
-

Algorithm 3 HTTPQ Verify

Require: envelope env = {AAD, C, T}

- 1: parse AAD → CBOR map; check schema
 - 2: rebuild AAD' → canonical CBOR and compare bytes
 - 3: derive keys, nonce per AAD
 - 4: decrypt and verify AEAD(C, T, AAD)
 - 5: recompute handle; must equal AAD.handle
 - 6: output payload if all checks pass
-

Verification error codes **MUST** correspond to Appendix G.

6 Ledger Notarization

Ledger notarization provides a verifiable mechanism to prove that a canonical B32K packet has been recorded in an immutable data structure. Each payload is represented by a **handle**, and multiple handles are aggregated into a Merkle tree whose root is subsequently anchored on a trusted ledger.

6.1 6.1 Handle and Digest Construction

For every canonical payload $N(x)$:

$$d = \text{BLAKE3-256}(N(x)) ,$$

$$\text{handle} = \text{BLAKE3-256}(\text{"B32K-HANDLE"} \parallel \text{version} \parallel \text{lane_id} \parallel d) .$$

The handle uniquely identifies both the payload and its metadata context. Implementations **MUST** recompute this handle during verification.

6.2 6.2 Merkle Aggregation

Handles are ordered lexicographically and aggregated using BLAKE3 in keyed mode to produce a Merkle root R :

$$h_i = \text{handle}_i, \quad R = \text{BLAKE3-256}(\text{"B32K-MERKLE"} \parallel h_0 \parallel h_1 \parallel \dots) .$$

Each inclusion proof consists of a sibling path $\{s_0, s_1, \dots, s_n\}$ allowing independent reconstruction of R . The resulting proof object is called an **O-1 receipt** (see Appendix C).

Verification. A verifier recomputes:

$$R' = \text{MerkleVerify}(h, \text{path}),$$

and compares $R' = R$. If equal, the packet is proven included in the notarized root.

6.3 6.3 Anchoring to a Ledger

Once R is produced, it **MUST** be anchored on an immutable ledger such as:

- A public blockchain transaction (e.g., Ethereum, Bitcoin, Solana);
- A permissioned consortium ledger;
- A timestamped notary log or append-only audit chain.

The anchor transaction **SHOULD** include:

- The Merkle root R ;
- The ledger domain identifier (URI);
- A timestamp and transaction identifier (txid);
- Optional metadata (e.g., signer public key, policy reference).

Persistence. Anchors **MUST** remain verifiable for the lifetime of the data they attest to. If the underlying ledger becomes unavailable, mirrored checkpoints **SHOULD** be maintained in redundant stores.

6.4 6.4 Receipts and Proof Objects

An O-1 receipt binds a single packet handle to its Merkle root and ledger anchor. It contains:

- handle;
- path (sibling hashes);
- root;
- timestamp;
- anchor_txid;
- ledger_uri.

Implementations **MUST** provide a reproducible verification path such that any independent verifier can recompute R and confirm inclusion using only the receipt and canonical payload.

6.5 6.5 Cross-Domain and NFT Integration (Non-Normative)

Handles and receipts may be referenced within external registries or token standards. A ledger anchor referencing a B32K handle may serve as a non-fungible token (NFT) proof of provenance or authorship, provided that:

- The NFT metadata contains the canonical handle and optional ledger_ref;
- The NFT's Merkle root matches the one verified via the O-1 receipt;
- Ownership semantics are defined outside this specification.

This linkage enables verifiable, cross-ledger provenance without revealing payload contents.

6.6 6.6 Security and Audit Notes

- BLAKE3 is chosen for its tree-hash structure, parallelism, and resistance to collision and length-extension attacks.
- Merkle proofs **MUST** be validated before any trust assertions.
- Anchors **MUST** be immutable once published.
- All receipt verification **MUST** be deterministic and side-effect-free.

Full schema and proof examples are given in [Appendix C](#).

7 Security Considerations

This section summarizes normative security requirements and implementation recommendations. The guidance herein applies to all B32K-compliant software, firmware, and ledger systems.

7.1 Key Management and Separation

- Each cryptographic role (signing, key exchange, encryption, ledger anchoring) **MUST** use a distinct keypair or derivation label.
- Key reuse across functions (e.g., using the same seed for Ed25519 and X25519) **MUST NOT** occur.
- Keys **MUST** be generated using a cryptographically secure random number generator (CSPRNG) providing at least 128 bits of entropy.
- Long-term identity keys **SHOULD** be stored in hardware-backed secure elements (e.g., HSM, TPM, or equivalent).

7.2 Randomness and Nonce Handling

- All ephemeral keys and nonces **MUST** be generated by a CSPRNG meeting or exceeding NIST SP 800-90A/B/C guidance.
- Nonces **MUST** be unique per encryption key. Reuse of a nonce with the same AEAD key immediately compromises confidentiality and integrity.
- Deterministic nonce derivation (Section B) **MAY** be used where strict reproducibility is required.

7.3 Replay Protection and Freshness

Replay protection is achieved via `packet_id` (UUIDv7) and monotonic `seq_no` fields in the HTTPQ envelope (Section 5). Receivers **MUST** enforce per-lane ordering and reject any packet with:

- a repeated or previously accepted `packet_id`;
- a `seq_no` lower than or equal to the last accepted sequence.

Systems maintaining persistent state **SHOULD** also track replay windows per cryptographic key epoch.

7.4 Hashing and Integrity Protection

- **BLAKE3** is the only normative hash for payload digest and Merkle aggregation.
- **XXH64** **MAY** be used for non-adversarial diagnostics only; it **MUST NOT** be used for integrity or signature inputs.
- AEAD algorithms **MUST** provide a 128-bit minimum authentication tag.

7.5 7.5 Glyphs, Fonts, and Human Interfaces

The B32K canonical alphabet defines 15-bit indices as authoritative; glyphs are advisory only. Implementations rendering B32K symbols in user interfaces **MUST** mitigate visual confusables and homoglyph attacks by:

- Restricting to a defined “safe plane” (see Appendix ??);
- Applying Unicode confusable filtering (UTR #36/#39);
- Normalizing all input to NFC before any encoding or signature step.

7.6 7.6 Post-Quantum Transition

The post-quantum hybrid suite (Section 4) provides transitional security against quantum adversaries. Implementations **MUST**

- Clearly label hybrid packets using the `alg_id` field;
- Include both classical and PQC shared secrets in HKDF derivation;
- Maintain forward compatibility for PQC-only suites when standardized.

Planar Permutation Layer (PPL). The PPL (Appendix D) introduces an optional entropy-mixing and obfuscation layer. It **MUST** be treated as experimental and **MAY** be enabled only in controlled research environments. The AEAD tag authenticates its output, ensuring no cryptographic weakening.

7.7 7.7 Implementation Best Practices

- All cryptographic operations **MUST** use constant-time libraries and side-channel-resistant primitives.
- Private keys **MUST** never appear in logs or telemetry.
- Canonicalization **MUST** occur before any hashing or signing.
- Receipt and Merkle proof verification **MUST** be pure, deterministic functions with no network dependencies.

Summary. Proper key separation, canonicalization, authenticated encryption, and ledger-anchored receipts together form the trust boundary of the B32K system. Any deviation from these constraints invalidates the security guarantees.

8 Compliance Mapping

Crosswalks to PCI DSS 4.0, HIPAA Security Rule, GDPR, NIST SP 800-53/161, SLSA.
Digest-only anchoring for privacy; TLS 1.3; AEAD; key lifecycle; deterministic $N(x)$ test vectors.

9 Compliance Mapping

This section summarizes how B32K aligns with major security, privacy, and governance frameworks. It is intended as an implementation crosswalk for auditors and compliance officers.

9.1 8.1 Standards Alignment Overview

Framework	Relevant B32K Controls and Features
PCI DSS v4.0	AEAD encryption (<i>ChaCha20–Poly1305</i>) and TLS 1.3 protect cardholder data in transit; per-session key rotation and nonce uniqueness meet requirement 3.6. Deterministic canonicalization ensures verifiable, tamper-evident records.
HIPAA Security Rule (45 CFR 164)	Canonicalization and authenticated encryption maintain integrity and confidentiality of electronic protected health information (ePHI). Ledger receipts enable audit and access accountability (164.312(a)(2)(iii)).
GDPR (EU 2016/679)	Digest-only ledger anchoring minimizes personal data processing. Anchors contain no plaintext identifiers. Optional consent metadata can be attached through external registries.
NIST SP 800-53 Rev. 5 / SP 800-161 Rev. 1	Maps to controls: SC-12 (cryptographic key establishment), SC-13 (cryptographic protection), AU-9 (audit protection), SR-11 (supply-chain integrity). Canonical test vectors allow reproducible control validation.
SLSA v1.0 (Software Supply-Chain Levels for Artifacts)	Canonicalization and BLAKE3 hashing provide provenance traceability for signed builds; O-1 receipts can embed in CI/CD attestations at SLSA Level 3+.

9.2 8.2 Privacy and Minimal Disclosure

Digest-only anchoring ensures no personal or sensitive content is published to the ledger. All canonical payloads are hashed before inclusion, and optional metadata (`ledger_ref`, `txid`) **MAY** be anonymized or omitted for private lanes.

9.3 8.3 Security Transport and Lifecycle Controls

- Transport confidentiality via TLS 1.3 or stronger;
- Authenticated encryption (AEAD) for payload confidentiality/integrity;
- Key rotation ≤ 24 h and per-lane key derivation (see Section 4);
- Deterministic $N(x)$ test vectors enable continuous conformance testing.

9.4 8.4 Assurance Summary

B32K’s canonicalization, authenticated transport, and ledger notarization collectively satisfy the core objectives of confidentiality, integrity, availability, and traceability under each referenced framework.

Non-Normative Note. Implementations **MAY** integrate additional compliance hooks (e.g., consent registries, pseudonymization, access logging) as required by regional or sectoral regulations.

A Canonicalization Test Vectors

This appendix defines deterministic reference vectors for validating the canonicalization process $N(x)$ and bit-packing of 15-bit indices. These vectors are normative for interoperability testing.

A.1 A.1 Scalar Examples

Input	Description	Canonical CBOR Hex ($N(x)$)
42	Unsigned integer	182A
"hello"	UTF-8 NFC string	6568656C6C6F
true	Boolean	F5
null	Null literal	F6

A.2 A.2 Structured Object Example

Input JSON:

```
{"lane":4,"version":"1.1.0","payload":[3,5,32767]}
```

Canonical CBOR (hex, deterministic byte order):

A3	# map(3)
64 6C616E65	# "lane"
04	
67 7061796C6F6164	# "payload"
83 03 05 19 7FFF	# array(3)
67 76657273696F6E	# "version"
65 312E312E30	# "1.1.0"

Canonical hex stream (no spaces):

A3646C616E6504677061796C6F6164830305197FFF6776657273696F6E65312E312E30

Digest. BLAKE3-256($N(x)$) = b8d9c894d44017fe7fb30ef6cc632d53e4ef36eb0d8a063fe8e7e2b3df0101a0

A.3 A.3 15-bit Packing Reference

Each index is a 15-bit unsigned integer $i \in [0, 32767]$. Indices are concatenated bitwise (big-endian) and padded to the next octet.

Example indices: [1, 2, 3]

$$1 = 000000000000001_2$$

$$2 = 000000000000010_2$$

$$3 = 000000000000011_2$$

Concatenation: 00000000 00000010 00000011 00000000

Packed hex (padded to 2 bytes boundary): 00020300

Round-trip.

$$\text{Unpack}(\text{Pack}([1, 2, 3])) = [1, 2, 3].$$

A.4 A.4 Signature Pre-image Template

The normative signature input is the canonicalization output plus metadata:

```
sig_input = CBOR({
  "version": "1.1.0",
  "lane_id": 4,
  "alg_id": 0x0101,
  "payload": N(x)
})
```

Signatures are computed over the exact CBOR bytes of `sig_input`. Whitespace, field order, or Unicode variations invalidate verification.

A.5 A.5 Conformance Validation Pseudocode

Algorithm 4 Round-Trip Canonicalization Test

Require: Implementation under test (IUT)

- 1: **for all** test_vector v in reference_suite **do**
 - 2: $b \leftarrow \text{IUT.CBOR_Canonicalize}(v.\text{input})$
 - 3: $\text{assert}(b == v.\text{expected_bytes})$
 - 4: $r \leftarrow \text{IUT.Decode}(b)$
 - 5: $\text{assert}(r == v.\text{input})$
 - 6: **end for**
 - 7: **return** "PASS"
-

A.6 A.6 Deterministic Output Property

Canonicalization and signing must be invariant under:

- JSON key order,
- UTF-8 normalization (NFC/NFD),
- numeric formatting (leading zeros, scientific notation),
- language or locale settings.

Any implementation producing divergent byte output for the same logical data structure is non-conformant.

B Cryptographic Suite Identifiers & Key Derivation Labels

This appendix defines the normative registry of algorithm identifiers (`alg_id`) and key derivation labels used throughout the B32K specification. Implementations **MUST** conform exactly to these identifiers for interoperability.

B.1 B.1 Algorithm Identifier Registry

Algorithm identifiers are 16-bit unsigned integers. The high-order byte indicates the major class; the low-order byte specifies the variant.

alg_id	Name	Description
0x0101	BASELINE	Ed25519 + X25519 + HKDF-SHA256 + ChaCha20-Poly1305 + BLAKE3.
0x0102	HYBRID-PQC	Ed25519 ML-DSA (Dilithium), X25519 ML-KEM (Kyber), HKDF-BLAKE3 + ChaCha20-Poly1305.
0x01F0--0x01FF	Experimental / PPL Variants	Reserved for Planar Permutation Layer or novel cipher constructions. Implementations MAY use these within private lanes.
0xFFFF	Reserved	Indicates an unsupported or unknown algorithm. Implementations MUST reject.

All identifiers are serialized as big-endian two-byte unsigned integers within the HTTPQ AAD (Section 5).

B.2 B.2 HKDF Domain-Separation Labels

HKDF labels are fixed ASCII strings used in extract-and-expand steps to ensure domain separation. They **MUST** be encoded exactly as shown (UTF-8, no null termination).

Label	Purpose
"B32K-KEX"	Key-exchange shared-secret expansion.
"B32K-SIGN"	Signature key derivation.
"B32K-PPL"	Planar Permutation Layer key seed.
"B32K-HANDLE"	Handle derivation (payload binding).
"B32K-MERKLE"	Merkle aggregation root computation.
"B32K-ENC"	Envelope AEAD encryption key expansion.

Implementations **MUST** apply these labels during HKDF-Extract or HKDF-Expand, using canonical case and order. Any deviation in label string or encoding invalidates key agreement consistency.

B.3 B.3 Deterministic Nonce Derivation

Nonces may be generated randomly or deterministically. The deterministic mode allows stateless nonce reconstruction and replay protection across lanes.

Definition.

$$\text{nonce}_{96} = \text{BLAKE3}_{96}(\text{"B32K-NONCE"} \parallel \text{lane_id} \parallel \text{packet_id} \parallel \text{seq_no})$$

Where:

- lane_id — 16-bit unsigned integer, big-endian;
- packet_id — 128-bit UUIDv7 (binary);
- seq_no — 64-bit unsigned integer, big-endian.

The result is truncated to 96 bits (12 bytes) and used directly as the AEAD nonce. Collisions are cryptographically improbable given unique packet_id per envelope.

B.4 B.4 Key-Separation Example

Example key hierarchy for a session under `alg_id = 0x0101`:

```
root_secret = HKDF-Extract(B32K-KEX, shared_secret)
enc_key     = HKDF-Expand(root_secret, "B32K-ENC", 32)
sign_key    = HKDF-Expand(root_secret, "B32K-SIGN", 32)
handle_key  = HKDF-Expand(root_secret, "B32K-HANDLE", 32)
```

Each key is independent by virtue of its label. Implementations **MUST NOT** derive further keys without explicit new labels.

B.5 B.5 Registry Management

The `alg_id` space `0x0200–0x7FFF` is reserved for future standardized algorithms; `0x8000–0xFFFF` for private and research use. Registry changes **MUST** be proposed via versioned specification updates.

C O-1 Receipt Schema and Merkle Proof Examples

The **O-1 Receipt** is a portable proof object linking a single B32K packet handle to its Merkle root and ledger anchor. Receipts provide offline verifiability of inclusion without exposing private payloads.

C.1 C.1 JSON Schema Definition

O-1 Receipts are JSON objects canonicalized under $N(x)$ (RFC 8785 + CBOR Canonical). The following JSON Schema is normative:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "B32K O-1 Receipt",
  "type": "object",
  "properties": {
    "version": { "type": "string", "pattern": "^1\\.\\.[0-9]+\\.\\.[0-9]+$" },
    "handle": { "type": "string", "minLength": 64, "maxLength": 64 },
    "path": { "type": "array", "items": { "type": "string", "minLength": 64 } },
    "root": { "type": "string", "minLength": 64, "maxLength": 64 },
    "anchor_txid": { "type": "string" },
    "ledger_uri": { "type": "string", "format": "uri" },
    "timestamp": { "type": "string", "format": "date-time" },
    "sig": { "type": "string", "minLength": 86, "description": "Base64url Ed25519/ML-DSA signature" }
  },
  "required": ["version", "handle", "path", "root", "ledger_uri", "timestamp"],
  "additionalProperties": false
}
```

All hex digests are lowercase, big-endian BLAKE3-256 outputs represented as 64-character hexadecimal strings.

C.2 C.2 Example Receipt

```
{
  "version": "1.1.0",
  "handle": "b8d9c894d44017fe7fb30ef6cc632d53e4ef36eb0d8a063fe8e7e2b3df0101a0",
  "path": [
    "119b1a4cc11a91db1c05ea00f4af57ee6027edb4513c9a890c153ef61fbe1b44",
    "9d14c62314de90f8ae741f2a7e7b5bc5d7a050253b9d8889da03c48e89a2fced"
  ],
  "root": "fa92a6b1c8932a2f9e41ecbf9a1a9d28d4e3df874cc2d0b5d71a3cbcc5042240",
  "anchor_txid": "0x82bc6f54a5e85d5c1e57f4d3bdf6b712ab90f53d",
  "ledger_uri": "https://ledger.example.org/tx/0x82bc6f54...",
  "timestamp": "2025-10-29T21:00:00Z",
  "sig": "YtS1m0ZsDK4JkoWSlULjF6d1hv0WfB8Pn1k..."
}
```

Canonical CBOR form. The canonicalized CBOR for this receipt begins with:

A8 67 76657273696F6E 65 312E312E30 66 68616E646C65 ...

Implementers **MUST** ensure byte-identical canonicalization prior to signing.

C.3 C.3 Verification Pseudocode

Algorithm 5 O-1 Receipt Verification

Require: canonical payload $N(x)$, receipt r

Ensure: boolean verified

- 1: $h \leftarrow \text{BLAKE3-256}(N(x))$
 - 2: $\text{assert}(h == r.\text{handle})$
 - 3: $R' \leftarrow \text{MerkleVerify}(h, r.\text{path})$
 - 4: $\text{assert}(R' == r.\text{root})$
 - 5: $\text{verify_sig}(r.\text{sig}, r.\text{root})$
 - 6: **return** true
-

If any assertion fails, the receipt **MUST** be rejected with the appropriate error code (Table C.1).

C.4 C.4 Error Codes

Code	Meaning
E01	Invalid BLAKE3 digest length.
E02	Handle mismatch – payload not included in receipt.
E03	Merkle path verification failed.
E04	Invalid ledger URI or anchor reference.
E05	Signature verification failed (Ed25519 / ML-DSA).
E06	Timestamp outside acceptable window.
E99	Schema validation error / malformed receipt.

C.5 C.5 NFT and Cross-Ledger Verification (Non-Normative)

A receipt **MAY** be bound to an NFT or token record by embedding its handle and root in the token metadata. To cross-verify:

1. Retrieve NFT metadata $\Rightarrow$ extract handle, root.
2. Verify $r.\text{handle} = \text{handle}_{\text{NFT}}$.
3. Confirm root appears in ledger block containing `anchor_txid`.
4. Validate chain inclusion proof (ledger $\rightarrow$ block $\rightarrow$ transaction).

Privacy Note. Because the O-1 Receipt references only digests, no private payload or personally identifying information is ever written to the ledger.

C.6 C.6 Deterministic Properties

- Every valid receipt yields the same Merkle root for identical handle sets.
- Signature and digest computations are independent of language or platform.
- Receipts are stable under canonical JSON $\leftrightarrow$ CBOR round-trips.

D Planar Permutation Layer (PPL)

The **Planar Permutation Layer (PPL)** is an optional, deterministic, and invertible transformation applied between canonicalization and encryption:

$$N(x) \xrightarrow{\text{PPL}} N'(x) \xrightarrow{\text{AEAD}} C.$$

It is intended primarily as a research mechanism for obfuscation and entropy-mixing, and is always authenticated by the outer AEAD envelope.

D.1 D.1 Formal Model

Given canonical payload $N(x)$ (a byte string of length L), we define:

$$\mathcal{D} = \{p_0, p_1, \dots, p_{n-1}\}, \quad p_i \in \mathbb{B}^m,$$

where m divides L , and $n = L/m$. Each p_i is called a *plane*.

Let K_p denote the HKDF-derived permutation key:

$$K_p = \text{HKDF_Expand}(\text{root_secret}, \text{"B32K-PPL"}, \ell_K).$$

From K_p , two pseudo-random sequences are derived via a PRF or PRP:

$$\sigma = \text{PRP}_{K_p}(0, 1, \dots, n-1), \quad \omega = \{\omega_0, \omega_1, \dots, \omega_{n-1}\},$$

where each $\omega_i \in [0, m-1]$ represents an angular rotation index.

The forward transform is defined as:

$$p'_i = \text{rotate}_{\omega_i}(p_{\sigma(i)}),$$

and the inverse as:

$$p_i = \text{rotate}_{-\omega_i}(p'_{\sigma^{-1}(i)}).$$

The rotation operator rotate_k performs a circular bit rotation within each plane. The permutation σ is a bijection over the index set $\{0, \dots, n-1\}$.

Determinism. For a given K_p and $N(x)$, the transform is deterministic and invertible. Any change to K_p , m , or n yields a distinct mapping.

D.2 D.2 Implementation Parameters

Parameter	Symbol	Typical Value / Range
Number of planes	n	8 – 64 (implementation-defined)
Plane size (bytes)	m	256 – 4096 (multiple of 16)
Rotation modulus	ω_i	Uniform random in $[0, 8m)$
Permutation key length	ℓ_K	32 bytes (BLAKE3 output)
PRF/PRP function	—	BLAKE3 or ChaCha20 counter mode

Implementations **MUST** ensure that all permutations and rotations are derived deterministically from K_p and are reproducible during decryption.

D.3 D.3 Authenticated Context

The PPL operates strictly within the authenticated domain of the outer AEAD. No additional tag is computed; any corruption in transformation state invalidates AEAD verification.

Security Note. Because the PPL is deterministic and publicly reversible with knowledge of K_p , it does not add cryptographic secrecy. Its purpose is entropy redistribution and pattern flattening before encryption.

D.4 D.4 Example Pseudocode

Algorithm 6 PPL Transform

Require: payload $N(x)$, permutation key K_p

Ensure: transformed payload $N'(x)$

- 1: Split $N(x)$ into planes $p_0..p_{n-1}$
 - 2: Generate σ, ω_i from PRP seeded by K_p
 - 3: **for all** i **do**
 - 4: $p'_i = \text{rotate}_{\omega_i}(p_{\sigma(i)})$
 - 5: **end for**
 - 6: Concatenate p'_i to obtain $N'(x)$
-

Algorithm 7 PPL Inverse Transform

Require: transformed payload $N'(x)$, permutation key K_p

Ensure: recovered payload $N(x)$

- 1: Regenerate σ, ω_i from K_p
 - 2: **for all** i **do**
 - 3: $p_i = \text{rotate}_{-\omega_i}(p'_{\sigma^{-1}(i)})$
 - 4: **end for**
 - 5: Concatenate p_i to recover $N(x)$
-

D.5 D.5 Quantum-Phase Security Research Path (Non-Normative)

The PPL conceptually models information as a set of “planes” possessing rotation and permutation degrees of freedom. This structure parallels quantum-mechanical state superposition in which amplitude and phase evolve under unitary transformations.

Future research may explore:

- Extending rotations ω_i to complex phase vectors;
- Embedding plane order σ as a decoherence-resistant basis selector in quantum or photonic encryption contexts;
- Mapping PPL states to qubit rotation operators (R_x, R_y, R_z) ;
- Analyzing entropy retention under reversible composite transforms.

Such exploration lies outside normative B32K scope but serves as a bridge between classical cryptography and experimental quantum-phase systems.

D.6 D.6 Summary

- PPL is deterministic, invertible, and AEAD-authenticated.
- No secrecy is derived from PPL alone.
- It provides a structured, mathematically well-defined space for entropy redistribution and potential future quantum alignment.

E Glossary and Symbol Index

This appendix consolidates key terms, symbols, and abbreviations used throughout the B32K Specification. It serves as a quick reference for implementers, auditors, and researchers.

E.1 E.1 Core Terms

Term	Definition
B32K	Base-32,768 encoding system using 15-bit indices as primary identifiers.
Index	Unsigned integer in $[0, 32767]$; authoritative identity of a symbol.
Glyph	Advisory Unicode rendering of an index; non-normative visual form.
Alphabet Table	Canonical mapping $A : \text{Index} \rightarrow \text{CodePoint}$ in Appendix ??.
Lane	16-bit domain-separation identifier isolating contexts or namespaces.
Version	Semantic version string (MAJOR.MINOR.PATCH) embedded in signatures and envelopes.
Anchor	Structured record binding version, lane, checksum, and signatures.
HTTPQ	Authenticated transport envelope with Public (AAD) and Private (AEAD) sectors.
Handle	Deterministic digest identifying a canonical payload and metadata set.

Ledger Anchor	Immutable record storing Merkle root or receipt hash on a trusted ledger.
O-1 Receipt	Compact proof object linking a handle to its Merkle root and ledger transaction.
PPL	Planar Permutation Layer — deterministic rotate-and-permute transform between canonicalization and AEAD.
Hybrid Suite	Combination of classical (Ed/X25519) and PQC (ML-DSA/KEM) primitives.
Canonicalization $N(x)$	Transformation to a byte-exact, platform-independent representation via UTF-8 NFC + canonical CBOR.
AEAD	Authenticated Encryption with Associated Data; ensures confidentiality and integrity.
Merkle Root	BLAKE3 aggregate digest summarizing a set of handles for notarization.
Receipt Path	Ordered list of sibling hashes reconstructing the Merkle root.
UUIDv7	Time-ordered 128-bit unique identifier for packet correlation.
Lane Key	Per-lane cryptographic context derived via HKDF.

E.2 E.2 Symbols and Constants

Symbol	Meaning	Typical Value / Domain
$N(x)$	Canonicalization function	CBOR-encoded bytes of x
d	Payload digest	BLAKE3-256 output (32 bytes)
handle	Payload + metadata digest	BLAKE3-256("B32K-HANDLE" ...)
R	Merkle root	BLAKE3-256 aggregate
σ	Permutation mapping	PRP output over plane indices
ω_i	Rotation offset for plane i	Uniform $\in [0, m-1]$
p_i	Plane i of payload	byte-segment of $N(x)$
K_p	PPL permutation key	HKDF-Expand("B32K-PPL")
m	Plane size (bytes)	256–4096
n	Number of planes	8–64
nonce ₉₆	Deterministic 96-bit AEAD nonce	From lane, packet_id, seq_no

E.3 E.3 Abbreviations

Abbrev.	Expansion
AEAD	Authenticated Encryption with Associated Data
CBOR	Concise Binary Object Representation (RFC 8949)
HKDF	HMAC-based Key Derivation Function (RFC 5869)
PQC	Post-Quantum Cryptography
PRF	Pseudo-Random Function
PRP	Pseudo-Random Permutation
RFC	Request for Comments (IETF standard)
TLS	Transport Layer Security (RFC 8446)

E.4 E.4 Notational Conventions

- `||` denotes byte-string concatenation.
- `||` within text indicates concatenation of algorithm results.
- Hex strings are lowercase and big-endian unless noted.
- All bit indices are big-endian (most-significant-bit first).
- All time fields use UTC ISO-8601 format.

E.5 E.5 Cross-Reference Index

Symbol / Term	Defined In	Referenced By
$N(x)$	§ 2.2, § 3	§ 4–7
handle	§ 5, § 6	§ 7, App C
alg_id	§ 4	App B, § 5
lane_id	§ 2, § 5	§ 4, App B
PPL	§ 4.3	App D
Merkle root R	§ 6	App C
O-1 receipt	§ 6	App C

Summary. The Glossary and Symbol Index form the authoritative semantic map for all identifiers in the B32K specification. Any divergence in terminology between implementations and this registry constitutes a non-conformant usage.

F Safe Rendering Plane (Non-Normative)

To prevent control-character injection and cross-plane ambiguity, implementers **SHOULD** adopt a single, printable Unicode plane as the default visual alphabet for human-readable contexts.

- Preferred block: **U+E000–U+E7FF** (Private-Use Area) mapped to B32K SAFE 0000 ... B32K SAFE 32767.
- Rendering font: monospaced or fixed-pitch; no combining marks.
- Prohibit: C0 and C1 controls (U+0000–U+001F, U+007F–U+009F), bidirectional override marks, and zero-width joiners.
- For UI display, implementations **SHOULD** apply Unicode confusable detection per [UTR #36](#) and [UTR #39](#).

This appendix is informative. Its purpose is to define a safe subset for interfaces, documentation, and transmission through systems that may otherwise filter or normalize control ranges.

Implementation Notes (Informative)

When mapping the canonical alphabet into the safe rendering plane:

- Each 15-bit index from 0x0000–0x7FFF **MUST** map deterministically to one code point in the private-use block.
- Glyph design **SHOULD** favor visual distinction and uniform advance width.
- Editors and rendering engines **MUST NOT** apply Unicode normalization (NFC/NFKC) across safe-plane glyphs.
- For font interoperability, authors **MAY** embed a b32k-safe.otf font defining one glyph per symbol.

Rationale

Control and formatting code points (e.g., U+0000–U+001F, U+200B–U+200F) can destabilize parsing, display, or hashing of canonical data. Restricting visual rendering to a printable subset mitigates truncation and spoofing risks while preserving human-readability for B32K-encoded material.

G HTTPQ Envelope Error Codes

This appendix defines a normative set of verification and processing errors for HTTPQ envelopes. Implementations **MUST** surface one of these codes when an operation fails during parse, canonicalization, AAD assembly, AEAD verification, or replay/nonce checks. Where applicable, remediation guidance is included.

G.1 G.1 Error Registry (Normative)

Code	Category	Meaning / Remediation	Severity
E10	ParseError	Malformed envelope framing or CBOR decode failure in Public Sector (AAD) or Private Sector (ciphertext). <i>Fix:</i> Re-send with valid canonical CBOR; check lengths and types.	hard
E11	CanonError	Canonicalization $N(x)$ mismatch versus declared schema (types, map order, definite lengths). <i>Fix:</i> Re-encode with UTF-8 NFC + canonical CBOR rules in §3.	hard
E12	AADMismatch	Local AAD reconstruction does not byte-match transmitted AAD. <i>Fix:</i> Ensure fields version, lane_id, alg_id, packet_id, seq_no, nonce, payload_len, handle follow §5.	hard
E13	AlgUnsupported	alg_id is unknown or disabled by policy. <i>Fix:</i> Negotiate a supported suite (App. B).	soft

Code	Category	Meaning / Remediation	Severity
E14	NonceReuse	Nonce collision detected for same key context (lane, sender). <i>Fix:</i> Rotate key or use deterministic nonces (App. B B.3).	hard
E15	ReplayDetected	Duplicate packet_id and/or non-monotonic seq_no. <i>Fix:</i> Ensure UUIDv7 uniqueness and increasing sequence.	hard
E16	StaleTimestamp	AAD timestamp outside acceptance window. <i>Fix:</i> Check clock sync; adjust skew policy.	soft
E17	KeyNotFound	No valid key material for lane/peer/epoch. <i>Fix:</i> Provision or rotate keys per §4.	hard
E18	AEADFail	AEAD tag verification failed (ciphertext/authentication corrupted). <i>Fix:</i> Re-send; investigate transport integrity/TLS.	hard
E19	HandleMismatch	handle in AAD does not equal local re-computation over $N(x)$. <i>Fix:</i> Recompute per §5 and §6.	hard
E1A	SizeMismatch	payload_len does not match decrypted plaintext length. <i>Fix:</i> Correct length prefix; re-package.	hard
E1B	PolicyViolation	Envelope violates local policy (lane blocked; suite disallowed; max size exceeded). <i>Fix:</i> Adjust policy or sender behavior.	soft
E1C	TLSRequired	Envelope arrived without TLS 1.3 where REQUIRED. <i>Fix:</i> Enforce TLS as per §5.	hard
E1D	ClockSkew	Skew exceeds limit but not conclusively stale. <i>Fix:</i> NTP sync; widen window if appropriate.	soft
E1E	Throttle	Rate limiting triggered for sender/lane. <i>Fix:</i> Backoff and retry later.	soft
E1F	InternalError	Unclassified internal failure; implementation should log details. <i>Fix:</i> Inspect server logs; file a bug.	hard

Severity *hard* indicates the envelope **MUST** be rejected; *soft* indicates the envelope **MAY** be retried after remediation or policy change.

G.2 G.2 Verification Decision Flow (Informative)

Algorithm 8 HTTPQ Verify

Require: Envelope env with AAD, nonce, ciphertext, tag

- 1: Parse CBOR(AAD, C) **or return** E10
 - 2: Canonicalize fields per $N(\cdot)$ **or return** E11
 - 3: Rebuild AAD' and compare bytes **or return** E12
 - 4: Check alg_id support **or return** E13
 - 5: Lookup key by (lane, peer, epoch) **or return** E17
 - 6: Validate nonce policy **or return** E14
 - 7: Enforce replay controls (packet_id, seq_no) **or return** E15
 - 8: Enforce timestamp window **or return** E16
 - 9: Require TLS 1.3 in transit (if policy) **or return** E1C
 - 10: Decrypt+Verify AEAD **or return** E18
 - 11: Verify handle = BLAKE3₂₅₆(...) **or return** E19
 - 12: Check length **or return** E1A
 - 13: Apply local policy limits **or return** E1B/E1E
 - 14: **return** ACCEPT
-

G.3 G.3 Mapping to Telemetry (Informative)

Implementations **SHOULD** log: lane_id, packet_id, alg_id, peer identity, error code, and a stable hash of AAD for privacy. Logs **MUST** exclude plaintext.

License: MIT + CC-BY 4.0
Repository: <https://github.com/rookinc/b32k-spec>
Version: 1.1.0 (Stable Draft — October 2025)
Contact: scott@rook.ca